

Submission Worksheet

Submission Data

Course: IT114-450-M2025

Assignment: IT114 Module 4 Sockets Part3 Challenge

Student: Mariano R. (mr822)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 10.00/10.00 (100.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 6/23/2025 5:54:31 PM

Updated: 6/23/2025 8:03:46 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT114-450-M2025/it114-module-4-sockets-part3-challenge/grading/mr822>

View Link: <https://learn.ethereallab.app/assignment/v3/IT114-450-M2025/it114-module-4-sockets-part3-challenge/view/mr822>

Instructions

- Overview Link: https://youtu.be/_029E_aBTfo
- 1. Ensure you read all instructions and objectives before starting.
- 2. Create a new branch from main called M4-Homework
 - 1. `git checkout main` (ensure proper starting branch)
 - 2. `git pull origin main` (ensure history is up to date)
 - 3. `git checkout -b M4-Homework` (create and switch to branch)
- 3. Copy the template code from here: [GitHub Repository - M4 Homework](#)
 - It includes Sockets Part1, Part2, and Part3. Put all into an M4 folder or similar if you don't have them yet (adjust package reference at the top if you chose a different folder name).
 - Make a copy of Part3 and call it Part3HW
 - Fix the package and import references at the top of each file in this new folder (Note: you'll only be editing files in Part3HW)
 - Immediately record to history
 - `git add .`
 - `git commit -m "adding M4 HW baseline files"`
 - `git push origin M4-Homework`
 - Create a Pull Request from M4-Homework to main and keep it open
- 4. Fill out the below worksheet
 - Each Problem requires the following as you work
 - Ensure there's a comment with your UCID, date, and brief summary of how the problem was solved
 - Code solution (add/commit periodically as needed)
 - Hint: Note how /reverse is handled
- 5. Once finished, click "Submit and Export"
- 6. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 - 1. `git add .`
 - 2. `git commit -m "adding PDF"`

3. git push origin M4-Homework

4. On Github merge the pull request from M4-Homework to main

7. Upload the same PDF to Canvas

8. Sync Local

1. git checkout main

2. git pull origin main

Section #1: (3 pts.) Challenge 1 - Coin Flip

Progress: 100%

≡ Task #1 (3 pts.) - Implement a Coin Flip Command

Progress: 100%

Details:

- `Client` must capture the user entry and generate a valid command per the lesson details
 - Command format must be `/flip`
- `ServerThread` must receive the data and call the correct method on `Server`
- `Server` must expose a method for the logic and send the result to everyone
 - The message must be in the format of
`<who> flipped a coin and got <result>` and be from the Server
- Add code to solve the problem (add/commit as needed)

🖼 Part 1:

Progress: 100%

Details:

Multiple screenshots are expected

1. Snippet of relevant code showing solution (with ucid/date comment) from `Client`
 - Should only need to edit `processClientCommands()`
2. Snippet of relevant code showing solution (with ucid/date comment) from `ServerThread`
 - Should only need to edit `processCommand()`
3. Snippet of relevant code showing solution (with ucid/date comment) from `Server`
 - Should only need to create a new method and pass the result message to `relay()`
4. Show 5 examples of the command being seen across all terminals (2+ Clients and 1 Server)
 1. This can be captured in one screenshot if you split the terminals side by side

```
break;
case "flip":
    // ucid: mr822 - 06/23/2025
    server.handleFlipCommand(this);
    wasCommand = true;
    break;

// added more cases/breaks as needed for other commands
default:
    break;
```

code 1

```
// ucid: mr822 - 06/23/2025
public void handleFlipCommand(ServerThread requester) {
    String result = Math.random() < 0.5 ? "Heads" : "Tails";
    String message = "Server: " + requester.getUsername() + " flipped a coin and got " + result;
    relay(null, message);
}
}
```

code 2

```
}
else if ("/flip".equalsIgnoreCase(text)) {
    // ucid: mr822 - 06/23/2025
    String[] commandData = { Constants.COMMAND_TRIGGER, "flip" };
    sendToServer(String.join(" ", commandData));
    wasCommand = true;
}
return wasCommand;
}
```

code 3

```

^Cmramos2001@DESKTOP-US958AD:~/IT114$ java M4.Part3HW.Client
Client Created
Client starting
Waiting for input
/connect localhost:3000
Client connected
Server: <User[22] connected>
Server: <User[23] connected>
Server: <User[24] connected>
/name Mariano
Server: Mariano has joined.
Server: Chris has joined.
Server: Ana has joined.
/flip
Server: Mariano flipped a coin and got Heads
Server: Chris flipped a coin and got Heads
Server: Ana flipped a coin and got Heads
Server: Chris flipped a coin and got Heads
/flip
Server: Mariano flipped a coin and got Heads

^Cmramos2001@DESKTOP-US958AD:~/IT114$ java M4.Part3HW.Client
Client Created
Client starting
Waiting for input
/connect localhost:3000
Client connected
Server: <User[25] connected>
Server: <User[26] connected>
Server: Mariano has joined.
/name Chris
Server: Chris has joined.
Server: Ana has joined.
Server: Mariano flipped a coin and got Heads
/flip
Server: Chris flipped a coin and got Heads
Server: Ana flipped a coin and got Heads
/flip
Server: Chris flipped a coin and got Heads
Server: Mariano flipped a coin and got Heads

^Cmramos2001@DESKTOP-US958AD:~/IT114$ java M4.Part3HW.C
Client Created
Client starting
Waiting for input
/connect localhost:3000
Client connected
Server: <User[24] connected>
Server: Mariano has joined.
Server: Chris has joined.
/name Ana
Server: Ana has joined.
Server: Mariano flipped a coin and got Heads
Server: Chris flipped a coin and got Heads
/flip
Server: Ana flipped a coin and got Heads
Server: Chris flipped a coin and got Heads
Server: Mariano flipped a coin and got Heads

```

testing



Saved: 6/23/2025 8:03:38 PM

Part 2:

Progress: 100%

Details:

Direct link to the file in the homework related branch from Github (should end in `.java`)

URL #1

[https://github.com/mramos822/mr822-](https://github.com/mramos822/mr822-IT114/blob/main/Homework/M4/Part3HW/Client.java)[IT114/blob/main-](https://github.com/mramos822/mr822-IT114/blob/main/Homework/M4/Part3HW/Client.java)[Homework/M4/Part3HW/Client.java](https://github.com/mramos822/mr822-IT114/blob/main/Homework/M4/Part3HW/Client.java)

URL

<https://github.com/mramos822/n>

URL #2

[https://github.com/mramos822/mr822-](https://github.com/mramos822/mr822-IT11450M4-Homework/M4/Part3HW/Server.java)

[IT11450M4-](https://github.com/mramos822/mr822-IT11450M4-Homework/M4/Part3HW/ServerThread.java)

[Homework/M4/Part3HW/Server.java](https://github.com/mramos822/mr822-IT11450M4-Homework/M4/Part3HW/ServerThread.java)



URL

<https://github.com/mramos822/n>



URL #3

[https://github.com/mramos822/mr822-](https://github.com/mramos822/mr822-IT11450M4-Homework/M4/Part3HW/ServerThread.java)

[IT11450M4-](https://github.com/mramos822/mr822-IT11450M4-Homework/M4/Part3HW/ServerThread.java)

[Homework/M4/Part3HW/ServerThread.java](https://github.com/mramos822/mr822-IT11450M4-Homework/M4/Part3HW/ServerThread.java)



URL

<https://github.com/mramos822/n>



Saved: 6/23/2025 8:03:38 PM

⇒ Part 3:

Progress: 100%

Details:

Briefly explain **how** the code solves the challenge (note: this isn't the same as **what** the code does)

Your Response:

The code solves the challenge by adding a new `/flip` command to the client-server chat system. When a client enters `/flip`, it sends a specially formatted command to the server. The server detects this command and triggers a method that randomly selects either "Heads" or "Tails." The result is then broadcast to all connected clients using the `relay()` method. This demonstrates command parsing, server-side logic branching, and synchronized message broadcasting, fulfilling the requirement for interactive server-triggered behavior.



Saved: 6/23/2025 8:03:38 PM

Section #2: (3 pts.) Challenge 2 - Private Message

Progress: 100%

≡ Task #1 (3 pts.) - Implement a Private Message Command

Progress: 100%

Details:

- **Client** must capture the user entry and generate a valid command per the lesson details
 - Command format must be `/pm <target id> <message>`
- **ServerThread** must receive the data and call the correct method on **Server**
- **Server** must expose a method for the logic
 - The message must be in the format of `PM from <who>: <message>` and be from the Server
 - The result must only be sent to the original sender and to the receiver/target
- Add code to solve the problem (add/commit as needed)

Part 1:

Progress: 100%

Details:

Multiple screenshots are expected

1. Snippet of relevant code showing solution (with ucid/date comment) from **Client**
 - Should only need to edit `processClientCommands()`
2. Snippet of relevant code showing solution (with ucid/date comment) from **ServerThread**
 - Should only need to edit `processCommand()`
3. Snippet of relevant code showing solution (with ucid/date comment) from **Server**
 - Should only need to create a new method and send the result message to just the sender and receiver
4. Show 3 examples of the command being seen across all terminals (3+ Clients and 1 Server)
 1. This can be captured in one screenshot if you split the terminals side by side
 2. Note: Only the sender and the receiver should see the private message (show variations across different users)

```
case "pm":
    // ucid: mr822 - 06/23/2025
    if (commandData.length >= 4) {
        long targetId = Long.parseLong(commandData[2].trim());
        String msg = String.join(" ", Arrays.copyOfRange(commandData, 3, commandData.length));
        server.handlePrivateMessage(this, targetId, msg);
        wasCommand = true;
    }
    break;
```

code 1

```
// ucid: mr822 - 06/23/2025
protected synchronized void handlePrivateMessage(ServerThread sender, long targetId, String msg) {
    String senderName = sender.getUsername() != null ? sender.getUsername() : "User[" + sender.getClientId() + "]";
    String formatted = "Server: PM from " + sender.getUsername() + ": " + msg;

    // Send to sender
    sender.sendToClient(formatted);

    // Send to receiver if they exist
    ServerThread receiver = connectedClients.get(targetId);
    if (receiver != null && receiver != sender) {
        receiver.sendToClient(formatted);
    }
}
```

code 2

```
wasCommand = true;
}
else if (text.startsWith("/pm")) {
    // ucid: mr822 - 06/23/2025
    String[] split = text.trim().split(" ", 3);
    if (split.length >= 3) {
        String[] commandData = { Constants.COMMAND_TRIGGER, "pm", split[1], split[2] };
        sendToServer(String.join(" ", commandData));
        wasCommand = true;
    }
}
```

```
return wasCommand;  
}
```

code 3

```
Closing output stream  
Closing input stream  
Closing connection  
Closed socket  
listenToServer thread stopped  
*Corvus2001@DESKTOP-US08AD: ~/IT11$ java M4.Part3HW.Client  
Client Created  
Client Starting  
Waiting for input  
/connect localhost:3000  
Client connected  
Server: User[24] connects  
/name Mariano  
Server: Mariano has joined.  
Server: Chris has joined.  
Server: Ana has joined.  
a  
User[24]: a  
User[22]: b  
User[22]: c  
/ps 23 Hello Chris  
Server: PS from Mariano: Hello Chris  
Server: PS from Ana: Hello Mariano
```

```
Closing input stream  
Closing connection  
Closed socket  
listenToServer thread stopped  
*Corvus2001@DESKTOP-US08AD: ~/IT11$ java M4.Part3HW.Client  
Client Created  
Client Starting  
Waiting for input  
/connect localhost:3000  
Client connected  
Server: User[23] connects  
Server: User[20] connects  
Server: Mariano has joined.  
/name Chris  
Server: Chris has joined.  
Server: Ana has joined.  
User[20]: a  
b  
User[23]: a  
User[22]: c  
Server: PS from Mariano: Hello Chris  
/ps 22 Hello Ana  
Server: PS from Chris: Hello Ana
```

```
*Corvus2001@DESKTOP-US08AD: ~/IT11$ java M4.Part3HW.Client  
Client Created  
Client Starting  
Waiting for input  
/connect localhost:3000  
Client connected  
Server: User[23] connects  
Server: User[24] connects  
Server: Mariano has joined.  
Server: Chris has joined.  
/name Ana  
Server: Ana has joined.  
/ps 22 Hello Mariano  
Server: PS from Ana: Hello Mariano  
User[20]: a  
User[22]: b  
c  
User[22]: c  
Server: PS from Chris: Hello Ana  
/ps 24 Hello Mariano  
Server: PS from Ana: Hello Mariano
```

testing

 Saved: 6/23/2025 8:03:09 PM

Part 2:

Progress: 100%

Details:

Direct link to the file in the homework related branch from Github (should end in `.java`)

URL #1

<https://github.com/mramos822/mr822-IT11/blob/main/M4/Part3HW/ServerThread.java>



URL

<https://github.com/mramos822/n>



URL #2

<https://github.com/mramos822/mr822-IT11/blob/main/M4/Part3HW/Server.java>



URL

<https://github.com/mramos822/n>



URL #3

<https://github.com/mramos822/mr822-IT11/blob/main/M4/Part3HW/Client.java>



URL

<https://github.com/mramos822/n>



 Saved: 6/23/2025 8:03:09 PM

Part 3:

Progress: 100%

Details:

Briefly explain **how** the code solves the challenges (note: this isn't the same as **what** the code does)

Your Response:

The code processes the /pm command by splitting it into sender, target, and message. The client passes this to the server in a structured format. The server thread detects the "pm" command and forwards the relevant data to the Server class. The Server then constructs a private message and sends it only to the sender and the specific receiver, avoiding other clients.

 Saved: 6/23/2025 8:03:09 PM

Section #3: (3 pts.) Challenge 3 - Shuffle Message

Progress: 100%

≡ Task #1 (3 pts.) - Implement a Shuffle Message Command

Progress: 100%

Details:

- `Client` must capture the user entry and generate a valid command per the lesson details
 - Command format must be `/shuffle <message>`
- `ServerThread` must receive the data and call the correct method on `Server`
- `Server` must expose a method for the logic and send the result to everyone
 - The message must be in the format of `Shuffled from <who>: <shuffled_message>` and be from the Server
- Add code to solve the problem (add/commit as needed)

Part 1:

Progress: 100%

Details:

Multiple screenshots are expected

1. Snippet of relevant code showing solution (with ucid/date comment) from `Client`
 - Should only need to edit `processClientCommands()`
2. Snippet of relevant code showing solution (with ucid/date comment) from `ServerThread`
 - Should only need to edit `processCommand()`
3. Snippet of relevant code showing solution (with ucid/date comment) from `Server`
 - Should only need to create a new method and do similar logic to `relay()`
4. Show 3 examples of the command being seen across all terminals (2+ Clients and 1 Server)
 1. This can be captured in one screenshot if you split the terminals side by side

```
break;
case "shuffle":
    // ucid: mr822 - 06/23/2025
    String shuffleText = String.join(" ", Arrays.copyOfRange(commandData, 2, commandData.length));
    server.handleShuffle(this, shuffleText);
```

```

        wasCommand = true;
        break;

// added more cases/breaks as needed for other commands
default:
    break;
}

```

code 1

```

// uuid: mr822 - 06/23/2025
protected synchronized void handleShuffle(ServerThread sender, String original) {
    char[] chars = original.toCharArray();
    java.util.Collections.shuffle(java.util.Arrays.asList(chars)); // won't work
    // Fix: shuffle via list conversion
    java.util.List<Character> charList = new java.util.ArrayList<>();
    for (char c : chars) charList.add(c);
    java.util.Collections.shuffle(charList);
    StringBuilder shuffled = new StringBuilder();
    for (char c : charList) shuffled.append(c);

    String senderName = sender.getUsername() != null ? sender.getUsername() : "User[" + sender.getClientId() + "]";
    String message = "Shuffled from " + senderName + ": " + shuffled;
    relay(null, message);
}
}

```

code 2

```

}
else if (text.startsWith("/shuffle")) {
    // uuid: mr822 - 06/23/2025
    String msg = text.replaceFirst("/shuffle", "").trim();
    String[] commandData = { Constants.COMMAND_TRIGGER, "shuffle", msg };
    sendToServer(String.join(",", commandData));
    wasCommand = true;
}
}

```

code 3

<pre> n(ForkJoinPool.java:1625) at java.base/java.util.concurrent.ForkJoinPool.run Worker(ForkJoinPool.java:1622) at java.base/java.util.concurrent.ForkJoinWorkerTh read.run(ForkJoinWorkerThread.java:165) Closing output stream Closing input stream Closing connection Closed socket ListensToServer thread stopped "CommandTrigger,shuffle,hello world" java MI.Part3W.Cli Client Created Client starting Waiting for input /connect localhost:3100 Client connected Server: 4000[22] connected Server: 4000[23] connected Server: Parlane has joined. /name Ana Server: Parlane has joined. Server: Ana has joined. Server: Shuffled from Ana: hello world /shuffle hi Ana how are you Server: Shuffled from Parlane: hi - an early res Server: Shuffled from Ana: test t each </pre>	<pre> JoinPool.java:1622) at java.base/java.util.concurrent.ForkJoinWorkerThread.run(orkJoinWorkerThread.java:165) Closing output stream Closing input stream Closing connection Closed socket ListensToServer thread stopped "CommandTrigger,shuffle,hello world" java MI.Part3W.Cli Client Created Client starting Waiting for input /connect localhost:3100 Client connected Server: 4000[22] connected Server: 4000[23] connected Server: Parlane has joined. /name Ana Server: Ana has joined. /shuffle hello world Server: Shuffled from Ana: hello world Server: Shuffled from Parlane: hi - an early res /shuffle this is a test Server: Shuffled from Ana: test t each </pre>	<pre> Server Starting Listening on port 3100 Waiting for next client Client connected Thread[0]: ServerThread created Waiting for next client Thread[22]: Thread starting Client connected Thread[0]: ServerThread created Waiting for next client Thread[23]: Thread starting Thread[23]: Received from my client: [cmd],name,Parlane Checking command: name Thread[22]: Received from my client: [cmd],name,Ana Checking command: name Thread[22]: Received from my client: [cmd],shuffle,hello wo rld Checking command: shuffle Thread[25]: Received from my client: [cmd],shuffle,hi Ana h ow are you Checking command: shuffle Thread[22]: Received from my client: [cmd],shuffle,this is a test Checking command: shuffle </pre>
---	---	--

testing

 Saved: 6/23/2025 8:03:19 PM

Part 2:

Progress: 100%

Details:

Direct link to the file in the homework related branch from Github (should end in `.java`)

URL #1
<https://github.com/mramos822/mr822-IT1141450M4-Homework/M4/Part3HW/ServerThread.java>

URL #2
<https://github.com/mramos822/mr822-IT1141450M4-Homework/M4/Part3HW/Server.java>

URL #3
<https://github.com/mramos822/mr822-IT1141450M4-Homework/M4/Part3HW/Client.java>

https://github.com/mramos822/n

https://github.com/mramos822/n

https://github.com/mramos822/n

https://github.com/mramos822/n

https://github.com/mramos822/n

https://github.com/mramos822/n

Saved: 6/23/2025 8:03:19 PM

≡

Part 3:

Progress: 100%

Details:

Briefly explain **how** the code solves the challenges (note: this isn't the same as **what** the code does)

Your Response:

The /shuffle command solves the challenge by sending a recognizable command from the client that the server parses and processes. The server shuffles the characters of the message, formats it with the sender's name, and broadcasts it to all clients, meeting the requirement to display a randomized message from the user to everyone.

Saved: 6/23/2025 8:03:19 PM

Section #4: (1 pt.) Misc

Progress: 100%

≡ Task #1 (0.33 pts.) - Github Details

Progress: 100%

Part 1:

Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history Following minimum should be present

M4 homework #3

11 Open

mramos822 wants to merge 5 commits into 822 from M4 homework

Commits

Commits

Commits

Files changed

Commit on Jun 19, 2024

added baseline files

adding M4 HW baseline files	mr822	06/23/2025	06/23/25	<>
Add Pac22 Server game solution	mr822	06/23/2025	06/23/25	<>
Add /pin and /flip command functionality - wcid: mr822 - 06/23/2025	mr822	06/23/2025	06/23/25	<>
Add /shuffle command functionality - wcid: mr822 - 06/23/2025	mr822	06/23/2025	06/23/25	<>

git



Saved: 6/23/2025 8:03:46 PM

Part 2:

Progress: 100%

Details:

Include the link to the Pull Request (should end in `/pull/#`)

URL #1

<https://github.com/mramos822/mr822-IT114-4453/commits>



URL

<https://github.com/mramos822/n>



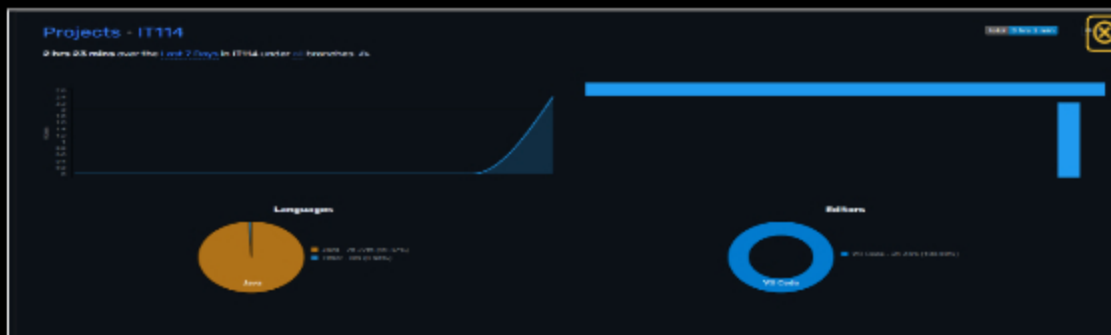
Saved: 6/23/2025 8:03:46 PM

Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click **Projects** and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



waka 1



58 mins M4/Part3HW/ServerThread.java
44 mins M4/Part3HW/Client.java
31 mins M4/Part3HW/Server.java
5 mins M4/Part1/Client.java
1 min M4/Part3HW/TextFX.java
58 secs .3-Homework/.gitignore:Zone.Identifier
46 secs M4/Part3HW/Constants.java

2 hrs 23 mins M4-Homework

waka 2



Saved: 6/23/2025 8:03:25 PM

≡ Task #3 (0.33 pts.) - Reflection

Progress: 100%

⇒ Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I learned how to handle client-server communication using sockets in Java, how to structure commands like /flip, /pm, and /shuffle, and how to route them through different layers of the application.



Saved: 6/23/2025 8:01:50 PM

⇒ Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part of the assignment was adding new commands in the client and routing them through the server thread. Once I understood the structure from existing commands like /reverse, it was straightforward to follow the same pattern for /flip, /pm, and /shuffle. The command parsing and sending data with `sendToServer()` became very intuitive after the first one.



Saved: 6/23/2025 8:02:09 PM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part of the assignment was understanding the problem itself at the beginning. It took time to figure out exactly how each command was supposed to work, what the expected output should be, and how the client and server were supposed to interact. Once I understood the structure and flow, the implementation became much easier.



Saved: 6/23/2025 8:02:41 PM